📖 Temas visuales avanzados

1. **Temas claros y oscuros**

- **Definición**: Flutter permite definir un **ThemeData** para personalizar colores, tipografías y estilos globales.
- **Teoría**: Se pueden configurar dos temas (`lightTheme` y `darkTheme`) y alternar entre ellos automáticamente o manualmente.
- **Ejemplo de uso**: Mejorar accesibilidad y experiencia del usuario según preferencias o configuración del sistema.

💻 Ejemplo: Tema claro/oscuro

```
import 'package:flutter/material.dart';

void main() {
  runApp(const ThemeApp());
}

class ThemeApp extends StatelessWidget {
  const ThemeApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Temas Avanzados',
      theme: ThemeData.light().copyWith(
        primaryColor: Colors.blue,
        textTheme: const TextTheme(bodyMedium: TextStyle(color: Colors.black)),
      ),
      darkTheme: ThemeData.dark().copyWith(
        primaryColor: Colors.deepPurple,
        textTheme: const TextTheme(bodyMedium: TextStyle(color: Colors.white)),
      ),
      themeMode: ThemeMode.system, // Cambia según configuración del dispositivo
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Tema dinámico')),
      body: const Center(child: Text('Este texto cambia con el tema')),
    );
```

```
  }
}
```

---

2. **Animaciones con Hero**

- **Definición**: `Hero` permite animar la transición de un widget entre pantallas.
- **Teoría**: Se usa para dar continuidad visual (ejemplo: una imagen que "vuela" de la lista al detalle).

💻 Ejemplo: Animación Hero

```
// En la pantalla de lista
GestureDetector(
  onTap: () {
    Navigator.push(
      context,
      MaterialPageRoute(builder: (_) => const DetailScreen()),
    );
  },
  child: Hero(
    tag: 'logo',
    child: FlutterLogo(size: 100),
  ),
);

// En la pantalla de detalle
Hero(
  tag: 'logo',
  child: FlutterLogo(size: 200),
);
```

👉 El widget con el mismo `tag` se anima automáticamente entre pantallas.

---

3. **Animaciones con AnimatedContainer**

- **Definición**: `AnimatedContainer` permite animar cambios de propiedades (color, tamaño, bordes) de forma automática.
- **Teoría**: Ideal para transiciones suaves sin necesidad de controlar manualmente la animación.

💻 Ejemplo: AnimatedContainer

```
class AnimatedBox extends StatefulWidget {
  const AnimatedBox({super.key});

  @override
  State<AnimatedBox> createState() => _AnimatedBoxState();
}
```

```
class _AnimatedBoxState extends State<AnimatedBox> {
  bool _big = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: AnimatedContainer(
          duration: const Duration(seconds: 1),
          width: _big ? 200 : 100,
          height: _big ? 200 : 100,
          color: _big ? Colors.blue : Colors.red,
          child: const Center(child: Text('Animado')),
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _big = !_big),
        child: const Icon(Icons.play_arrow),
      ),
    );
  }
}
```

---

4. **Animaciones con Lottie**

- **Definición**: Lottie permite usar animaciones vectoriales exportadas desde After Effects o similares.
- **Teoría**: Se integran con el paquete `lottie` y ofrecen animaciones profesionales listas para usar.

💻 Ejemplo: Lottie

```
import 'package:flutter/material.dart';
import 'package:lottie/lottie.dart';

class LottieExample extends StatelessWidget {
  const LottieExample({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: Lottie.network(
          'https://assets9.lottiefiles.com/packages/lf20_touohxv0.json',
          width: 200,
          height: 200,
          repeat: true,
        ),
      ),
    );
```

```
    }
  }
```

🎯 Resumen:

1. **Personalizar temas globales** (colores, tipografías, estilos).
2. **Implementar modo claro/oscuro** adaptado al sistema.
3. **Usar animaciones Hero** para transiciones visuales entre pantallas.
4. **Aplicar AnimatedContainer** para cambios suaves de propiedades.
5. **Integrar Lottie** para animaciones vectoriales profesionales.